Python Modules and Packages

import, Custom Modules, Packages, __name__, and the Standard Library

1. Introduction

As programs grow, it becomes impractical to keep all code in a single file. Python's module system lets you split code into separate files that can be reused across different programs, and lets you take advantage of a vast ecosystem of pre-written functionality through the standard library and third-party packages. This module explains what a module is, how to import code from one file into another, how to create your own modules and organize them into packages, and how to use several of the most useful modules in Python's standard library.

Understanding modules is essential for writing maintainable, non-trivial software: it allows logical separation of concerns (e.g. keeping database code separate from user-interface code) and prevents you from reinventing functionality that Python already provides.

2. Learning Objectives

- Explain what a module is and why code is organized into modules.
- Import whole modules and specific names using import and from ... import.
- Create and use a custom module file.
- Organize related modules into a package using __init__.py.
- Explain the purpose of __name__ and the if __name__ == "__main__": idiom.
- Use useful standard library modules: math, os, pathlib, json, and collections.

3. Concept Explanations

3.1 What Is a Module?

A module is simply a Python file (`.py`) containing definitions — functions, classes, and variables — that can be imported and reused in other files. Python itself ships with a huge collection of built-in modules known as the **standard library**, covering everything from mathematics to file paths to date/time handling.

3.2 import and from ... import

The `import` statement loads a module and makes its contents accessible through the module's name. `from ... import` pulls specific names directly into the current namespace, so they can be used without the module prefix.

```
import math
print(math.sqrt(16))          # 4.0 - accessed via the module name

from math import sqrt, pi
print(sqrt(16))                # 4.0 - used directly
```

```
print(pi)                       # 3.141592653589793

import math as m                # aliasing for shorter names
print(m.floor(4.7))             # 4
```

3.3 Creating Custom Modules

Any `.py` file can act as a module for another file in the same directory (or on Python's module search path). For example, if you create a file `calculator.py` containing function definitions, another script can import it by filename (without the .py extension).

```
# --- calculator.py ---
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

# --- main.py ---
import calculator

print(calculator.add(3, 4))
print(calculator.subtract(10, 6))

# Alternatively:
from calculator import add
print(add(5, 5))
```

3.4 Packages and __init__.py

A package is a directory containing multiple related modules, grouped together and imported using dotted names. Traditionally, a package directory contains an `__init__.py` file, which marks the directory as a package and can run initialization code or expose selected names when the package is imported. In modern Python (3.3+), packages can technically work without `__init__.py` (as 'namespace packages'), but including it remains standard practice for clarity and compatibility.

```
my_package/
    __init__.py
    shapes.py
    colors.py

# usage from outside the package:
from my_package import shapes
from my_package.colors import get_hex_code
```

3.5 __name__ and if __name__ == "__main__":

Every Python module has a built-in variable `__name__`. When a file is run directly (e.g. `python script.py`), Python sets `__name__` to `"__main__"` for that file. When the same file is imported as a module from elsewhere, `__name__` is instead set to the module's own name. This lets you write code that only runs when the file is executed directly, and stays quiet when it is imported — extremely useful for including test code or a demo alongside reusable functions.

```
# --- calculator.py ---
```

```
def add(a, b):
    return a + b

if __name__ == "__main__":
    # this block only runs when calculator.py is executed directly,
    # NOT when it is imported by another file
    print("Running self-test:")
    print(add(2, 2))
```

3.6 Useful Standard Library Modules

Module	Purpose	Example
math	Mathematical functions and constants	math.sqrt(25), math.ceil(4.1), math.pi
os	Interacting with the operating system	os.getcwd(), os.listdir('.'), os.path.join('a','b')
pathlib	Object-oriented, cross-platform file paths	Path('data') / 'file.txt', path.exists()
json	Reading and writing JSON data	json.dumps(data), json.loads(text)
collections	Specialized container data types	Counter(items), defaultdict(list), namedtuple

```
import json
data = {"name": "Alice", "age": 30}
text = json.dumps(data)         # convert dict -> JSON string
print(text)
restored = json.loads(text)     # convert JSON string -> dict
print(restored["name"])

from collections import Counter
votes = ["cat", "dog", "cat", "fish", "dog", "cat"]
print(Counter(votes))           # Counter({'cat': 3, 'dog': 2, 'fish': 1})

from pathlib import Path
p = Path("reports") / "summary.txt"
print(p, p.suffix, p.exists())
```

4. Syntax Summary

```
import module_name
import module_name as alias
from module_name import name1, name2
from module_name import name as alias

if __name__ == "__main__":
    # code that runs only when this file is executed directly
    ...

package/
    __init__.py
    module_a.py
    module_b.py
```

5. Code Examples

Example 1: Using math for Geometry

```
import math

radius = 5
area = math.pi * radius ** 2
circumference = 2 * math.pi * radius
print(f"Area: {area:.2f}, Circumference: {circumference:.2f}")
```

Example 2: Listing Files with os and pathlib

```
import os
from pathlib import Path

current_dir = os.getcwd()
print("Current directory:", current_dir)

folder = Path(".")
for item in folder.iterdir():
    print(item.name, "- file" if item.is_file() else "- folder")
```

Example 3: A Reusable Custom Module

```
# --- string_utils.py ---
def is_palindrome(text):
    cleaned = text.lower().replace(" ", "")
    return cleaned == cleaned[::-1]

if __name__ == "__main__":
    print(is_palindrome("Was it a car or a cat I saw"))

# --- main.py ---
from string_utils import is_palindrome
print(is_palindrome("level"))   # True
```

6. Step-by-Step Explanation

Walking through Example 3 (Custom Module) line by line:

1. `string_utils.py` defines a reusable function, `is_palindrome`, that can be imported anywhere.
2. The `if __name__ == "__main__":` block contains a quick self-test that only executes when `string_utils.py` is run directly.
3. In `main.py`, `from string_utils import is_palindrome` imports just the function, without triggering the self-test block, because `__name__` is `"string_utils"` in that context, not `"__main__"`.
4. Calling `is_palindrome("level")` from main.py reuses the exact same logic without duplicating any code.

7. Common Mistakes

■ **Common Mistake:** naming a personal script the same as a standard library module (e.g. math.py), which shadows the real module and causes confusing import errors.

■ **Common Mistake:** forgetting the if __name__ == "__main__": guard, so demo or test code runs unexpectedly every time the module is imported elsewhere.

■ **Common Mistake:** using from module import * , which pollutes the namespace and makes it unclear where a name came from.

■ **Common Mistake:** forgetting that Python caches imported modules — editing a module file will not affect an already-running interactive session without reloading.

■ **Common Mistake:** mixing up os.path string-based paths with pathlib.Path objects inconsistently within the same project.

8. Best Practices

✔ **Best Practice:** import only what you need with from module import specific_name, or import the whole module and use dotted access for clarity.

✔ **Best Practice:** always guard executable demo/test code with if __name__ == "__main__": in reusable modules.

✔ **Best Practice:** prefer pathlib over manual string concatenation for file paths — it is safer and more cross-platform.

✔ **Best Practice:** group related modules into a well-named package rather than leaving many loose files in one folder.

9. Practice Questions

1. What is the difference between import module and from module import name?
2. What value does __name__ hold when a file is imported versus when it is run directly?
3. Why is from module import * generally discouraged?
4. What is the purpose of __init__.py in a package directory?
5. Name three functions from the math module and describe what each does.

10. Mini Coding Exercises

1. Create a custom module geometry.py with functions for the area of a circle, square, and triangle, then import and use it in another script.
2. Write a script that uses the json module to save a dictionary to a file and then load it back.
3. Write a script that uses collections.Counter to find the most common word in a sentence.
4. Write a script that uses pathlib to list all .txt files in the current directory.
5. Create a package with two modules (e.g. converters.py and validators.py) and an __init__.py, then import and use functions from both in a main script.

11. Interview / QC Questions

1. How does Python locate a module when you write import some_module? Explain the role of sys.path.
2. What is the difference between a module and a package?
3. Why is the if __name__ == "__main__": idiom considered a best practice in reusable Python scripts?
4. What are namespace packages, and how do they differ from traditional packages with __init__.py?
5. How would you avoid circular imports between two modules that need to reference each other?

12. Summary

This module explained how Python organizes code into reusable files (modules) and directories of related files (packages), and how import and from ... import bring that code into another file. You learned to create your own modules, structure packages with __init__.py, and use the if __name__ == "__main__": idiom to separate reusable logic from run-only-when-executed-directly demo code. Finally, you were introduced to several essential standard library modules — math, os, pathlib, json, and collections — that solve common problems without requiring you to write the logic yourself. Together with functions, data structures, and OOP, modules complete the toolkit needed to build organized, maintainable, real-world Python applications.

End of Python_Modules — this concludes the Python LMS Fundamentals track.